

C PROGRAMMING

DEBUGGING TOOLS

Project Report

Submitted by	Rudra Pratap Singh
Sap ID / Global ID	590032811
Course / Subject	C Programming
Date	26 August 2026

Contents

1. Fundamentals of Errors and Debugging
2. Classification of Errors in C
3. GNU Debugger (GDB) - Overview and Core Commands
 - Case 1: Syntax and Compilation Errors
 - Case 2: Compiler Warnings (-Wall, -Wextra)
 - Case 3: printf-Based Debugging (Logical Bug)
 - Case 4: Debugging a Logical Error using GDB
 - Case 5: Debugging Functions (next vs. step)
 - Case 6: Runtime Errors using Backtrace
 - Case 7: Array Out-of-Bounds using GDB
 - Case 8: Segmentation Faults (SIGSEGV) using GDB
4. Systematic Debugging Strategy
5. Conclusion

1. Fundamentals of Errors and Debugging

Error	A mistake or blunder made by the programmer in the source code.
Bug	A flaw or fault in a program that causes an incorrect, unexpected or unintended result.
Debugging	The systematic process of finding, analyzing and removing bugs or defects from a computer program.

Important point: A compiler checks C syntax and type rules, but it cannot understand the programmer's intention. Therefore, logically wrong code can compile successfully and still produce incorrect output.

2. Classification of Errors in C

Error Type	Description	Primary Cause	Example
Syntax Error	Violates C grammar rules.	Missing semicolon, unmatched braces, keyword typo.	int a = 10 (missing ;)
Compilation / Type Error	Uses incompatible types or missing declarations.	Type mismatch or undeclared variable.	int *p = "hello";
Linker Error	Object files cannot be combined successfully.	Declared function not defined or missing library.	undefined reference to main
Runtime Error	Program terminates abnormally while running.	Division by zero or invalid memory access.	int x = 10 / 0;
Logical Error	Program runs but produces the wrong result.	Wrong formula or incorrect loop condition.	i < 5 instead of i <= 5

3. GNU Debugger (GDB) - Overview and Core Commands

- GNU stands for "GNU's Not Unix!" and is an open-source operating system project initiated by Richard Stallman in 1983.
- GDB (GNU Debugger) is the standard debugger for the GNU software system. It can inspect variables, set breakpoints and analyze crashes.
- Compile with debugging symbols using -g before starting a GDB session.

Installation and compilation commands

```
# Install GDB on a Linux system
sudo apt update
sudo apt install gdb

# Compile with debugging symbols and useful warnings
cc -g -Wall -Wextra program.c -o program

# Launch GDB
gdb ./program
```

Command	Shorthand	Purpose
break <line/func>	b	Set a breakpoint.
run	r	Start the program.
next	n	Execute next line; step over function calls.
step	s	Execute next line; step into function calls.
print <var>	p	Print a variable or expression.
display <var>	disp	Automatically show a value after each step.
continue	c	Resume until next breakpoint or completion.
list	l	Show source around the current line.
info locals	info loc	List local variables.
backtrace	bt	Show the current call stack.
info breakpoints	i b	Show active breakpoints.
quit	q	Exit GDB.

Case 1: Syntax and Compilation Errors

Purpose. Demonstrate how the compiler reports invalid C syntax, then apply the smallest correction and recompile.

Initial code - sample1.c (contains the error)

```
#include <stdio.h>

int main()
{
    int a = 10
    printf("%d\n", a);
    return 0;
}
```

Observed error	expected ',' or ';' before 'printf'
Cause	The declaration <code>int a = 10</code> is missing its terminating semicolon.
Fix	Add <code>;</code> after <code>10</code> , then compile and execute again.

Snippet 1 - Compilation error followed by corrected compilation and execution

Terminal - Syntax Error

```
rudra@rudra:~$ cc sample1.c
sample1.c:5:5: error: expected ',' or ';' before 'printf'

# After adding the missing semicolon
rudra@rudra:~$ cc sample1_fixed.c -o sample1
rudra@rudra:~$ ./sample1
10
```

Corrected code - sample1_fixed.c

```
#include <stdio.h>

int main()
{
    int a = 10;
    printf("%d\n", a);
    return 0;
}
```

Result: The compiler accepts the corrected program and execution prints 10.

Case 2: Compiler Warnings (-Wall, -Wextra)

A warning means the code is syntactically legal but potentially suspicious. Warnings do not necessarily stop compilation, so warning flags are useful for finding bugs early.

Initial code - sample2.c

```
#include <stdio.h>

int main()
{
    int a;
    printf("Hello\n");
    return 0;
}
```

Standard build	The program compiles and prints Hello.
With warnings	-Wall -Wextra reports that variable a is unused.
Fix used	Remove the unused variable because it is not needed by this program.

Snippet 1 - Standard compilation, warning-enabled compilation and clean rebuild

```
Terminal - Compiler Warnings

rudra@rudra:~$ cc sample2.c
rudra@rudra:~$ ./a.out
Hello

rudra@rudra:~$ cc -Wall -Wextra sample2.c
sample2.c:5:9: warning: unused variable 'a' [-Wunused-variable]

# Minimal fix: remove the unused variable
rudra@rudra:~$ cc -Wall -Wextra sample2_fixed.c -o sample2
rudra@rudra:~$ ./sample2
Hello
```

Corrected code - sample2_fixed.c

```
#include <stdio.h>

int main()
{
    printf("Hello\n");
    return 0;
}
```

Why the flags matter: -Wall enables a broad set of commonly useful warnings; -Wextra enables additional warnings not covered by -Wall.

Case 3: printf-Based Debugging (Logical Bug)

Purpose. Use printf statements to expose loop execution and the changing value of sum. The intended total is $1 + 2 + 3 + 4 + 5 = 15$.

Initial code - sample3.c

```
#include <stdio.h>

int main(void)
{
    int i, sum = 0;

    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: entering iteration i = %d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
    }

    printf("Final Sum = %d\n", sum);
    return 0;
}
```

Expected	15, by adding 1 through 5.
Actual	10, because the loop reaches i = 0,1,2,3,4 and never processes 5.
Debug clue	The DEBUG lines show the exact value of i during each iteration.
Fix	Start at 1 and continue while i <= 5.

Snippet 1 - Value tracing makes the missing i = 5 iteration visible

Terminal - printf Debugging

```
rudra@rudra:~$ cc sample3.c -o sample3
rudra@rudra:~$ ./sample3
DEBUG: entering iteration i = 0
DEBUG: sum updated to = 0
DEBUG: entering iteration i = 1
DEBUG: sum updated to = 1
DEBUG: entering iteration i = 2
DEBUG: sum updated to = 3
DEBUG: entering iteration i = 3
DEBUG: sum updated to = 6
DEBUG: entering iteration i = 4
DEBUG: sum updated to = 10
Final Sum = 10

# Corrected loop: i = 1; i <= 5; i++
rudra@rudra:~$ cc sample3_fixed.c -o sample3_fixed
rudra@rudra:~$ ./sample3_fixed
Final Sum = 15
```

Corrected code - sample3_fixed.c

```
#include <stdio.h>

int main(void)
{
    int i, sum = 0;

    for(i = 1; i <= 5; i++)
        sum = sum + i;

    printf("Final Sum = %d\n", sum);
    return 0;
}
```

Case 4: Debugging a Logical Error using GDB

Initial code - sample4.c

```
#include <stdio.h>          // Line 1
int main()                 // Line 2
{                           // Line 3
    int i, sum = 0;         // Line 4
    for(i = 1; i < 5; i++) // Line 5 - logical bug
    {                       // Line 6
        sum = sum + i;      // Line 7
    }                       // Line 8
    printf("Sum = %d\n", sum); // Line 9
    return 0;               // Line 10
}
```

Expected output	15 (1 + 2 + 3 + 4 + 5)
Actual output	10 (1 + 2 + 3 + 4)
Breakpoint	Line 7: sum = sum + i;
Finding	The loop terminates when i reaches 5, so the body does not run for 5.
Fix	Change i < 5 to i <= 5.

Snippet 1 - GDB breakpoint, variable display and repeated continue

● ● ● GDB - Logical Error Trace

```
rudra@rudra:~$ cc -g sample4.c -o sample4
rudra@rudra:~$ gdb ./sample4
(gdb) list
(gdb) break 7
Breakpoint 1 at 0x1149: file sample4.c, line 7.
(gdb) run
Starting program: ./sample4
Breakpoint 1, main () at sample4.c:7
7      sum = sum + i;
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.
Breakpoint 1, main () at sample4.c:7
1: i = 2
2: sum = 1
(gdb) continue
...
1: i = 4
2: sum = 6
(gdb) continue
Sum = 10
[Inferior 1 (process) exited normally]
```

Snippet 2 - Corrected build and execution

● ● ● Terminal - Corrected Logical Error

```
# Fix: change i < 5 to i <= 5
rudra@rudra:~$ cc -g sample4_fixed.c -o sample4_fixed
rudra@rudra:~$ ./sample4_fixed
Sum = 15
```

Corrected code - sample4_fixed.c

```
#include <stdio.h>

int main()
{
    int i, sum = 0;
    for(i = 1; i <= 5; i++)
        sum = sum + i;

    printf("Sum = %d\n", sum);
    return 0;
}
```

Case 5: Debugging Functions (next vs. step)

This case does not contain a source-code bug. It demonstrates two GDB commands used while debugging function calls: next executes the call without entering the function body, while step enters the function for line-by-line inspection.

Typed code - sample5.c

```
#include <stdio.h>

int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}

int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```

Snippet 1 - Stepping over the function with next

```
● ● ● GDB - Stepping Over with next

rudra@rudra:~$ cc -g sample5.c -o sample5
rudra@rudra:~$ gdb ./sample5
(gdb) break main
(gdb) run
(gdb) next          # Moves to result = square(n);
(gdb) next          # Executes square(n) completely
(gdb) print result
$1 = 25             # Result obtained without entering square()
```

Snippet 2 - Stepping into the function with step

```
● ● ● GDB - Stepping Into with step

(gdb) break main
(gdb) run
(gdb) next          # Moves to result = square(n);
(gdb) step          # Enters square(x)
square (x=5) at sample5.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
(gdb) print result1
$2 = 25
```

next

Executes the current line and completes a called function without transferring GDB focus into it.

step	Transfers GDB focus into the called function so its statements and local values can be inspected.
Observed value	square(5) produces result1 = 25 and main receives result = 25.

Case 6: Runtime Errors using Backtrace (backtrace / bt)

Purpose. Use GDB to identify a division-by-zero crash and inspect the sequence of function calls that led to it.

Initial code - sample6.c

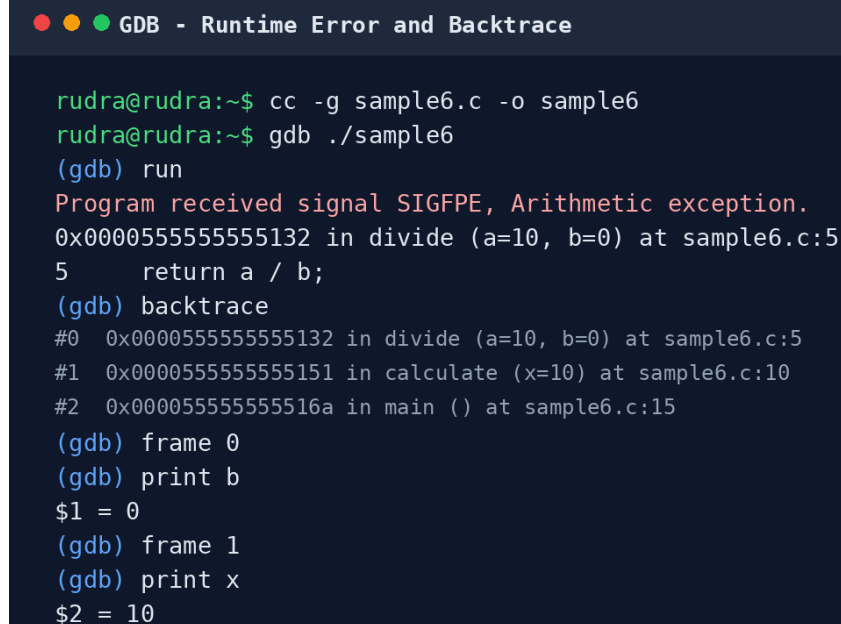
```
#include <stdio.h>

int divide(int a, int b)
{
    return a / b; // Division by zero when b = 0
}

int calculate(int x)
{
    return divide(x, 0); // Passes 0 as divisor
}

int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

Snippet 1 - SIGFPE, backtrace and frame inspection



```
GDB - Runtime Error and Backtrace

rudra@rudra:~$ cc -g sample6.c -o sample6
rudra@rudra:~$ gdb ./sample6
(gdb) run
Program received signal SIGFPE, Arithmetic exception.
0x000055555555132 in divide (a=10, b=0) at sample6.c:5
5     return a / b;
(gdb) backtrace
#0  0x000055555555132 in divide (a=10, b=0) at sample6.c:5
#1  0x000055555555151 in calculate (x=10) at sample6.c:10
#2  0x00005555555516a in main () at sample6.c:15
(gdb) frame 0
(gdb) print b
$1 = 0
(gdb) frame 1
(gdb) print x
$2 = 10
```

Frame #0	divide: the crash occurs at return a / b with b = 0.
Frame #1	calculate: this function called divide(x, 0).

Frame #2	main: this started the call sequence by calling calculate(10).
Root cause	The divisor passed to divide is 0.

Corrected code - sample6_fixed.c

```
#include <stdio.h>

int divide(int a, int b)
{
    return a / b;
}

int calculate(int x)
{
    return divide(x, 2); // Demonstration fix: non-zero divisor
}

int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

Snippet 2 - Corrected execution

```
Terminal - Corrected Runtime Error

# Demonstration fix: use a non-zero divisor (2)
rudra@rudra:~$ cc -g sample6_fixed.c -o sample6_fixed
rudra@rudra:~$ ./sample6_fixed
Result: 5
```

Correction note: The supplied tutorial identifies $b = 0$ as the fault. This report uses 2 only as a simple non-zero demonstration value for the corrected run.

Case 7: Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked automatically at runtime. Using an index beyond the declared range can produce undefined behavior or memory corruption.

Initial code - sample7.c

```
#include <stdio.h>

int main()
{
    int a[5];
    int i;

    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]); // Line 8
    }

    return 0;
}
```

Declared array	int a[5] gives valid indices 0, 1, 2, 3 and 4.
Bug	The condition <code>i <= 5</code> also allows <code>i = 5</code> .
GDB inspection	Printing <code>&a[5]</code> demonstrates that the program is attempting to address an element outside the valid array range.
Fix	Use <code>for(i = 0; i < 5; i++)</code> .

Snippet 1 - GDB address inspection and corrected execution

```

GDB - Array Bounds Inspection

rudra@rudra:~$ cc -g sample7.c -o sample7
rudra@rudra:~$ gdb ./sample7
(gdb) break 8
(gdb) run
(gdb) print &a[0]
$1 = (int *) 0x7fffffff9c0
(gdb) print &a[4]
$2 = (int *) 0x7fffffff9d0
(gdb) print &a[5]
$3 = (int *) 0x7fffffff9d4
# Valid indices are 0..4; index 5 is out of bounds.

# Fix: change i <= 5 to i < 5
rudra@rudra:~$ cc -g sample7_fixed.c -o sample7_fixed
rudra@rudra:~$ ./sample7_fixed
10
20
30
40
50
rudra@rudra:~$

```

Corrected code - sample7_fixed.c

```

#include <stdio.h>

int main()
{
    int a[5];
    int i;

    for(i = 0; i < 5; i++)
    {
        scanf("%d", &a[i]);
    }

    return 0;
}

```

Case 8: Segmentation Faults (SIGSEGV) using GDB

A segmentation fault occurs when a program attempts to access memory that it is not permitted to access. The example dereferences a NULL pointer, so GDB reports SIGSEGV at the invalid write.

Initial code - sample8.c

```
#include <stdio.h>

int main()
{
    int *p = NULL; // Pointer is 0x0
    *p = 10;       // Invalid write -> segmentation fault
    printf("%d\n", *p);
    return 0;
}
```

Snippet 1 - SIGSEGV, pointer inspection and corrected run

```
GDB - Segmentation Fault

rudra@rudra:~$ cc -g sample8.c -o sample8
rudra@rudra:~$ gdb ./sample8
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x000055555555139 in main () at sample8.c:6
6      *p = 10;
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
Cannot access memory at address 0x0

# Corrected program points p to valid memory
rudra@rudra:~$ cc -g sample8_fixed.c -o sample8_fixed
rudra@rudra:~$ ./sample8_fixed
10
```

Crash line	*p = 10;
GDB value	p = (int *) 0x0
Meaning	The pointer does not refer to a valid writable object.
Fix	Create a real int object x and point p to &x before writing through p.

Corrected code - sample8_fixed.c

```
#include <stdio.h>

int main()
{
    int x;
    int *p = &x; // p points to valid memory
    *p = 10;     // Valid write
    printf("%d\n", *p);
    return 0;
}
```

Result: The corrected program writes through a valid pointer and prints 10.

4. Systematic Debugging Strategy

1	Observe expected output versus actual output.
2	Reproduce the issue consistently.
3	Compile with useful flags: -Wall -Wextra -g.
4	Isolate suspicious sections of code.
5	Inspect program state using printf or GDB breakpoints.
6	Apply one minimal fix.
7	Recompile and re-test.
8	Verify using edge cases and boundary inputs.

5. Conclusion

The cases in this report show that debugging is a systematic activity rather than random trial and error. Compiler diagnostics identify syntax problems, warning flags expose suspicious code, printf statements reveal program state, and GDB provides breakpoints, stepping, variable inspection and backtraces for logical and runtime failures. A reliable workflow is to reproduce the problem, inspect the evidence, apply one minimal correction, rebuild and verify the result.

Submission checklist: Each case contains hand-typed C code, one or two terminal-style snippets covering compilation/execution/debugging, a short explanation of the problem and fix, and corrected code where a source-code correction is required.